Introduce myself

b252ceca2aabe6c409879da65b17f79ad4347269db79cb3cf3300ef07b1f6f77

Slides 1：The Problem
Given 600x600 permittivity map，1X2 source location，ouput real and imagery part of the fields

1. 怎么系统的找到最好NN架构

sudo apt install python-pip

sudo apt update

sudo apt install python3-pip

ls -la

cd .local

pwd

export PATH='/home/viska/.local/bin:$PATH"

~/.bashrc

export PATH='/home/viska/.local/bin:$PATH'

Log:

Why ANR and vit

1. Data has negative values
2. Need more data?
3. Fatal Information loss

2 approoches:
Top-down, Button-up

https://aistudio.baidu.com/projectdetail/7682679

交大：用深度学习解微分方程-理解神经网络
https://www.bilibili.com/video/BV1Aq4y1P7gN/

【深度学习实践】第二节：使用神经网络求解偏微分方程
https://www.bilibili.com/video/BV1VT411E7sY/?

【深度学习实践】第七节：神经网络ODE实现发动机模型降阶
https://www.bilibili.com/video/BV1sU4y1z7Qb/

1. Generate green's function wave,.
2.

1. Physics-Informed Deep Neural Operator Networks https://arxiv.org/pdf/2207.05748

2. https://www.science.org/doi/10.1126/sciadv.abi8605
Pin message
3. https://proceedings.mlr.press/v202/hao23c/hao23c.pdf

Pin message
Eric Yue Ma
12:53 AM

https://arxiv.org/abs/2403.03542

1) pip install --upgrade tnr
2) tnr status
3) nvidia-smi

Reason why it is slow and why switching to a100 will not help much:
他们用的driver（torch，accelerate）都是非常老的版本（torch = 1.11.0 vs 2.4.0，用lightning训练不需要再额外加accelerate）根本没法很好的utilize GPU。
：`torch==2.4.0` 包含了大量的性能优化、内存管理改进和新功能。例如，PyTorch 2.x 引入了加速器支持的优化、更好的调度器和调优技术，这些都会直接提高训练速度。

新版本包含了很多性能改进：优化的 CUDA 支持、更高效的算子实现以及更好的分布式训练支持。如果你使用的是较新的硬件（如最新的 GPU），升级 PyTorch 可能会提高训练速度。

我甚至需要downgrade CUDA 才能跑通代码。

我暂时没动任何代码，而是hotfix了wandb，EMdataloader，output image writer function
这一技术俗称“石山雕花”

A. Results

- 4 images processed
- Training completed for 1000 epochs
-

Check for detailed results:

- Running time: View on Weights & Biases (Wandb)
- Parameters: View on Weights & Biases (Wandb)
- Loss curve: View on Weights & Biases (Wandb)

Link to Wandb project: https://wandb.ai/kronosai-kronosai/anr-em

B. Challenges and Solutions

1. Performance Bottlenecks:

- Outdated software stack: The current ANR code relies on older versions of PyTorch and CUDA, hindering performance on newer hardware (A10, A100, H100). Latest PyTorch versions offer significant improvements (memory management, CUDA support, distributed training) leading to faster training times.
- Limited hardware utilization: Even with an A100 GPU, GPU utilization remains low (0%). Upgrading hardware alone wouldn't solve the issue. To achieve compatibility, CUDA had to be downgraded further limiting performance.

  Solution: Downgrade software stack to:

  - PyTorch: 2.4.0 (Note: Lambda Labs dropped 1.10 support) -> 1.11.0
  - Torchvision: 0.19 -> 0.12.0
  - CUDA: 12.2 -> 11.3

2. Hotfixes Implemented:

- Wandb: additional logging on top of the original ANR loggers.
- EMDataloader: Implemented to handle EM data.
- Output Image Writer Function: Hardcoded to the training function.

These temporary fixes to outdated code, often referred to as "shi山雕花", ensure system functionality until core code improvements can be implemented.

3. Integer Source Embedding Requirement:

The source embedding tensor requires integer values for x0 and y0 (e.g., tensor[2, 3] instead of tensor[2.1, 3.2]). ‘’’tensor[x0, y0] = 2.0’’’

C. Lessons Learned: Importance of Rapid Validation

Before investing in large-scale training on expensive GPUs, rapid model validation is crucial to identify functionality issues and prevent resource waste.

<u>EM Data Validation Process:</u>

A step-by-step validation process was developed to ensure EM data accuracy:

1. Input/Output Field Validation: Verify model capability to overfit a single image with both input and output fields within a few iterations (e.g., 10 epochs). Gradually increase the number of samples (2, 4, 8, ... 128).

2. Triplet Validation: Verify model capability to overfit triplets (permittivity, source embedding, real field) within a few iterations. Gradually increase the number of samples (2, 4, 8, ... 128).
3. Origin Input/Output Validation: Verify model capability to overfit (permittivity+source → fields) within a few iterations. Gradually increase the number of samples (2, 4, 8, ... 128).

All steps can be efficiently implemented using a Tesla T4 GPU without A100s.

D: Lessons Learned: Why Vanilla transformer doesn't work?

```
| NVIDIA-SMI 535.129.03             Driver Version: 535.129.03   CUDA Version: 12.2     |
|-----------------------------------------+----------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|                                         |                      |               MIG M. |
|=========================================+======================+======================|
|   0  NVIDIA A10                     On  | 00000000:08:00.0 Off |                    0 |
|  0%   39C    P0              56W / 150W |   5674MiB / 23028MiB |      0%      Default |
|                                         |                      |                  N/A |
+-----------------------------------------+----------------------+----------------------+

+---------------------------------------------------------------------------------------+
| Processes:                                                                            |
|  GPU   GI   CI        PID   Type   Process name                            GPU Memory |
|        ID   ID                                                             Usage      |
|=======================================================================================|
|    0   N/A  N/A     41134      C   python                                    5662MiB |
+---------------------------------------------------------------------------------------+
```

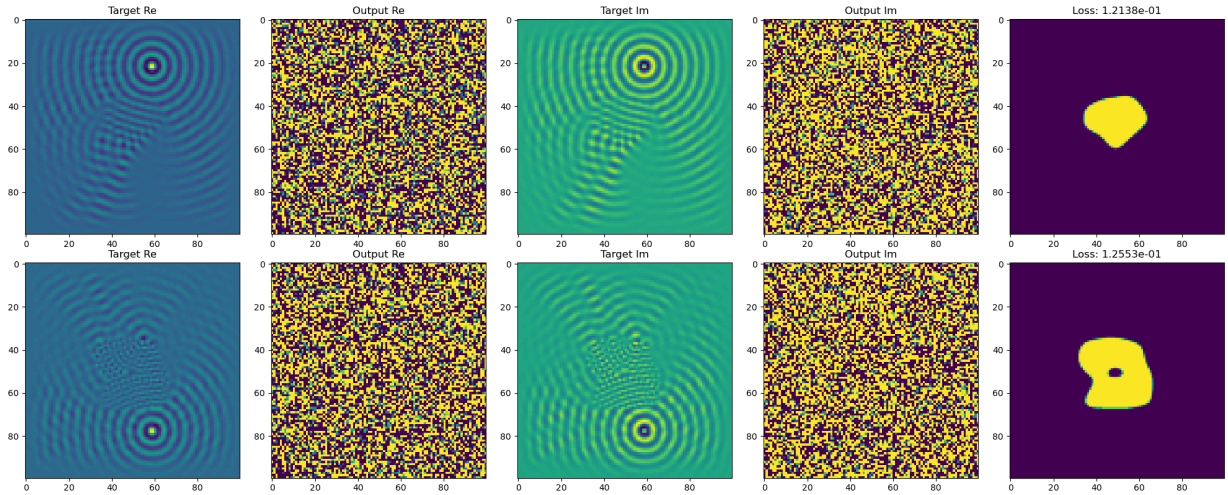
```
pip install --user wandb
pip install --user --force-reinstall wandb
export PATH="$HOME/.local/bin:$PATH"
source ~/.bashrc

pip install torch==1.11.0+cu113 torchvision==0.12.0+cu113 torchaudio==0.11.0+cu113
--extra-index-url https://download.pytorch.org/whl/cu113

y
```

Wandb login
Key: d420a14ad27943c26db6f51c996070bf6294d0a0

Aug 29, 2024

1, ANR attention
2. Source → image

Vanilla vision transformer not working: (100ep, input fields and output fields)



https://colab.research.google.com/drive/1y0RmVf_IPQi9Uj8wxLLCiODHXs1LoSsc?usp=sharing#scrollTo=XetGiSRiARn_

`Swin2SRImageProcessor`

`https://huggingface.co/docs/transformers/main/en/tasks/image_to_image`

1. 可以用swin直接实验

Aug 28, 2024

```
Host lambda
    User ubuntu
    HostName 150.136.146.50
```

```
    IdentityFile C:\Users\viska\.ssh\id_rsa
```

1. **Adapt ANR**

- **(1.1) 使用CelebA复现结果:**
  - **障碍:** 目前的代码结构比较复杂，尤其是在理解数据加载器如何读取数据以及数据结构方面还不清晰。我需要和ANR的作者安排一次会议，以澄清这些问题。
- **(1.2) 将ANR接口适配Eric的数据:**
  - 计划修改ANR的接口，以处理Eric提供的特定数据。

2. **继续修改我的代码**

- **(2.1) 确保能在单张图片上过拟合:**
  - 我已经在单张图片上实现了过拟合。
- **(2.2) 尝试在两张图片上过拟合:**
  - 结果表现较差，可能是因为permittivity的嵌入效果不佳。
  - **(2.2-vit-1) 改变ViT的patch size实验:** 将ViT的patch大小从5改为1后，效果有所提升。
  - **(2.2-vit-2) 实验:** [尚未明确].
  - **(2.2-unfold-1) 用unfold替代ViT:** 直接展开并添加source embedding，结果导致空间信息严重丧失（loss=1e-3），直接添加source embedding可能是不合理的。
  - **(2.2-unfold-2) 分离permittivity和source的嵌入:** 使用两对KV分别处理permittivity和source的嵌入，单张图片的结果有所改进（loss=1e-4），这表明这种方法可能是正确的。
- **(2.3) 修改损失函数:**
  - 使用梯度和拉普拉斯损失函数的效果比MSE更差，生成的图像相似度过高。
- **(2.4-att-1) 修改注意力机制:**
  - 我正在尝试更新V，而不仅仅是Q。
- **(2.4-att-2) 使用ANR的注意力机制:**
  - 我正在探索使用ANR的注意力机制。
- **(2.4) 最佳模型在128张图片上的测试:**
  - 我已经在128张图片上测试了最佳模型，loss=1e-3。

下一步计划:
- 对于**ANR**，我会优先解决数据处理方面的问题，并安排必要的会议。
- 对于**代码修改**，我会继续优化有前景的方法，例如分离嵌入，同时继续实验不同的损失函数和注意力机制，以提升模型性能。

Aug 27, 2024

pip install gdown
Gdown link
sudo apt-get install unzip

ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAABgQC9YBVFBcYELTV90kRBuR+aol2Fnk5H9aO1HkRg
FnxDAaMQvgvk4kGVcdz/OpJ7s+2u54+f8qXXxWpyZEjI7mH9GXq89aC1umpfqiOYP2/tX6LnKV
HQWGoEEjWeyvqDb+Yv5puCjrJmS8jI1ZTZeYsCoRDbD0x7rOmv14jlO2k85Ieu/g0bLhK04Bd83
gjt6WTKVvMQ69ouaqhVBoaLP8ZRmi/Fubpi5yIHWiKd3ttK/5NQf/XAeryqBeORmw+ce0DsfTA8
uf3OREFipNh5xnhGM4PMP3kB/ZPpL9qBGlqWEUuDFfoNQw6wz6CQ23P7JBefxKaJGtPKF9Y
bMFXNQRz0zPlZe46mY8eGeHizCVkEhS6ZBdoPSS87neMSHyOKo2xwmU4JPt6K3m5z/W5R
WcnL/JalOjmRexVT0jjMQZfRMV5JMl/iLiFj5ZgkqG5ncKqLmKn3Fn1cAmxNW/lvYW0yOP2dNA
RySf/M4otLMnUOSBTVisKNHbiGc4k4gOSzIDk= viska@DESKTOP-SL2GRA8

```
Host tnr
    User ubuntu
    HostName 34.134.230.145
    IdentityFile C:\Users\viska\.ssh\id_rsa
```

Background (current bottleneck):

The attention mechanism's memory usage becomes a critical issue due to its O(n^2) complexity. Specifically, with a 600x600 input, the computational complexity scales to 600^4 = 10^8, which is significantly larger than what is typically manageable (e.g., a sequence length of 512 or even the recent advancements supporting up to 32k <<10^8 tokens). This exponential growth in required resources is the primary challenge I am working to address.

Current Status:

Here is a summary of the approaches I've been exploring:

1. **Direct Dimensionality Reduction (600x600 → 512):**
    - **Pros:** This approach simplifies the input and reduces computational overhead.
    - **Cons:** The major drawback is the significant information loss, making it difficult for the model to **overfit a single image**.
2. **Chunked Attention (simpletr_chuck.py) and Sparse Attehtion (sparse_att.py) :**

- **Pros:** I attempted to implement chunked attention and sparse attention to manage the memory usage more effectively.
- **Cons:** Unfortunately, this approach still results in **memory issues,** suggesting that even with chunking, the **complexity remains too high for the available resources.**

3. **Dimensionality Reduction via Encoder:**
   - **Pros:** Using an encoder to reduce the input dimensions to 512 seems promising.
   - **Cons:** Doesn't make sense to encode the coordinate which contains no info.
4. **Coordinate Grid with Interpolation (upsample.py):**
   - **Pros:** I am currently testing the use of a 30x30 coordinate grid, which I plan to upsample using interpolation at the output stage. This method could balance computational efficiency with sufficient detail retention.
   - **Cons:** I am still in the process of validating whether this approach can successfully **overfit a single image.**
   - If not, I will try convolution transpose 2D to upsample.

Tag 0.0.1
Loss ep 100 - 0.00

pip install tnr --upgrade

```
sudo fallocate -l 20G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
sudo swapon -s
```

Aug 24, 2024 ANR

S3:
sudo apt-get update
sudo apt-get install awscli

aws configure

access key:
AKIAQFLZDMX5CL226FBY

secret access key:
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

Region: us-west-1
Output format…: h5

s3://cakronosaibucket/2024_08_data_set_02a/02_filter_r10_20240803-191649.h5

aws s3 cp s3://cakronosaibucket/2024_08_data_set_02a/ . --recursive

https://intellipaat.com/blog/aws-s3-cli-commands/

tnr run python run.py

Aug 23, 2024

Problems:

1. Tnr C:\Users\viska>ssh 34.71.141.216
2. viska@34.71.141.216: Permission denied (publickey).
3. **ubuntu@34.134.230.145**

4. ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQC2rnZr2zhOiZc/kCK1+6sb/+mRfO3kQecaRPGMjtF0fcwfhREZGN22nHQig+RKIUDwXB4wgBdUIHoGW8a5M3X5zxNhxkZ8vRL1YONUdOz5yzUTMmAATuFvH1Y2JL5KS/mrSAPzhWmMJgqVJWOtGjxRXEsejyQaa/0rrCXOypCjy/+DS2/zW5GPiyiBu33E4w+xMWH/4hzELlcnQekgMnFEFds3lS2jB6Wc6luC7yb5tm5Z7WdM6PQe6lqhIXZC+XACxiz1wyv8/Fr+npRSvWUWsMMP497QeoHzRAjbbfsR5J1lhHdZehAmyYdzVckyou3ToD/Me/q6rkPaamT5bXGJ/WkOByHUMWlFcroMJdp/LYFkYgeGKAVQiOG+n5Tjn/AJSDQ9858OHKSZCouF4M+KHjcKrVqfBTs2FNu7SESjix/qSAOrVZBfUKGT+kqbFhy0CMd3+V3PVeIwN1Hlf+jGLmVkDWT1Ol6m46/SGG24qerUjYYs66XZ8rTeOA1+FAs= viska@DESKTOP-DCES5VO

TNR token

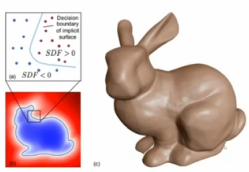
AMf-vBz7ZJcygyv30DG_GSta8VvTJQTLNGpf-p_sAeAxDj6UlZZzEj_qIBQsEX4K0KBOQC0kyems-Opop6GTWqodHk1DiCThhOt7fRBND90JklJgjlszGzARkWqxmD5oGKsUYRpGI_YAvaCm4Z6EgLJvM04eXSMsKLF0NWsi6MIhbkzNtoZ1QH2HUL-CHWfM6OWPZkITbVGLhKNN-HXtlkTBZ

iY6uunseBQbwGyr6pG2Wi6zdE0TAQS-YurfUPKcYxTCBRYbWSRQe9HhoEteXfGHOqEFY9Si
V061zx9PT_D3cLLZSwJzY8jHndpTaqhlx7VFLQ4ds1pQO66SBYUyg_8qoSB5mZ-JSOX2kov5
WuHZE8dDd_N7Y7dAslPTWbnSzKqG3ffHBgxMqj4Fgj043i6VJ18Lz49ssE9fZW0nAOvIZlpHTy
KlHR4

AMf-vBz7ZJcygyv30DG_GSta8VvTJQTLNGpf-p_sAeAxDj6UlZZzEj_qIBQsEX4K0KBOQC0kye
ms-Opop6GTWqodHk1DiCThhOt7fRBND90JklJgjlszGzARkWqxmD5oGKsUYRpGI_YAvaCm4Z
6EgLJvM04eXSMsKLF0NWsi6MIhbkzNtoZ1QH2HUL-CHWfM6OWPZkITbVGLhKNN-HXtlkTBZ
iY6uunseBQbwGyr6pG2Wi6zdE0TAQS-YurfUPKcYxTCBRYbWSRQe9HhoEteXfGHOqEFY9Si
V061zx9PT_D3cLLZSwJzY8jHndpTaqhlx7VFLQ4ds1pQO66SBYUyg_8qoSB5mZ-JSOX2kov5
WuHZE8dDd_N7Y7dAslPTWbnSzKqG3ffHBgxMqj4Fgj043i6VJ18Lz49ssE9fZW0nAOvIZlpHTy
KlHR4

Neural Fourier Transforms

Understanding Implicit Neural Representations with Itzik Ben-Shabat



$$\mathcal{S} = \{x \mid \Phi(x) = 0\}$$

Implicit representation

[Physics Informed Neural Networks (PINNs) [Physics Informed Machine Learning]]

https://www.bilibili.com/video/BV1NL411k7FV/

Aug 22, 2024 Goal: Try to overfit ANR

1. Fully understand the underlying framework of ANR, SIREN.
2. Then carefully check whether ANR is correctly implemented.
3. Aim to overfit.

Aug 21, 2024 Start Training ANR

1. Signal too small, mean(Ez) = 1e-5

2. Attempt: Normalized input, Results: Not promising
3. Next: Check NN architecture and implementation.

Aug 20, 2024 Research on Architectures

1. Implicit Neural Rrepresentations:
    a. "隐式定义": 这意味着我们不是直接存储数据的每一个点,而是用一个函数来间接描述整个信号。
2.

SIREN: Implicit Neural Representations with Periodic Activation Functions (Paper Explain…
Implicit Neural Representations with Periodic Activation Functions

1. If your NN models your signal very well, it should also models the derivatives of the signal very well.

Aug 18, 2024 Code Explore

https://arxiv.org/pdf/2407.15355

Eric's Siren code:
2024-08 explore_siren_for_2D_EM.ipynb

The code that generated the data:
2024-07 fdtdz test.ipynb

Fei's code to load the data:
attached.

Aug 17, 2024 Setup

1. tnr: 34.71.141.216
    a. Tnr: quickstart
    b. Tnr device (T4, A100)
    c. Isa.pub: ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQC6VaTRDV8ioBemBNmJ6i0qfvPtbGe3mnQJuhb/kF4PYgiJgVhsgnQysuJU/hBTByRFb964bR+8jgYjWypk/P7BovsLkqHeYujbzMwpkQt+10JPNy08gN7LEkxJfpNoomDOBfKjHnVX3cc8JaNeNndAoA0URDm5qbqKaIV78KJCe+5RzL2V7dA8jWSWIdB81bLBToL31NUSP3e6dHgvjrxF

Mst3ARciMAKZGfuNMNohYdywYAYwSWFzTyzwUoIL3nqgMu+NIfIV9rz8WGkM
d5+mRNvbuV5BmTXRB9g+EXc/tct6Hphv/eMDL4+e2y11LixktqVEp/W/pL4QhbP
xC6FkT3pwzLyZOUWWguMCgi6nE0t0IQsNRnI8SeqjecFkXj6uIyrforKdMV+qwyZ
yugO6GPSBPfiEISjBdY8RFmQ5hE+cM2mOjrYMIZB9BLvCtujUu3L2OA3FoXe7
ScBUOPdaxvFU8bH7upobUrqe1CnjSNEdGObXG0/efPZ1QcarwCs=
viska@WIN-N8GQP9AUNLG

2. AWS: https://011528267258.signin.aws.amazon.com/console
    a. Username: Viska Pwd 1fY!..
3. Poseidon
    a. Paper:
    b. Model: https://huggingface.co/camlab-ethz/Poseidon-T
    c. Notes: https://docs.google.com/document/d/1qeP7DxPG3EbiQgLfAIm5TeMBUFPhnpmWBPclATW2hCo/edit?usp=sharing
    d.
    e.

Ref

Slides:
https://docs.google.com/presentation/d/1AsERfY3Df7xbuTQO-H3ri8b-GfwoBqMDFnFJzyYBe7I/edit?usp=sharing

Relevant papers:
https://arxiv.org/pdf/2405.17527
https://arxiv.org/pdf/1907.04502
https://arxiv.org/pdf/2405.13998
https://arxiv.org/pdf/2405.19101
https://arxiv.org/pdf/2402.12365
https://arxiv.org/pdf/2403.03542
https://arxiv.org/pdf/2209.10098
https://pubs.acs.org/doi/full/10.1021/acsphotonics.3c00156
https://dl.acm.org/doi/pdf/10.1145/3648506
https://www.science.org/doi/full/10.1126/sciadv.abi8605
https://arxiv.org/pdf/2203.01248
https://www.nature.com/articles/s41566-020-0685-y
https://www.nature.com/articles/s41578-020-00260-1
https://pubs.aip.org/aip/app/article/7/1/011301/2835095/MaxwellNet-Physics-driven-deep-neural-network
https://arxiv.org/abs/2406.12904